

Activity 9: Architecture and Traceability Documentation

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K5, A5

Estimated time: 75 minutes

Objective

Create design documentation that maps user specifications to architecture, interfaces and tests.

Deliverable

C4 diagrams, two ADRs, API contract and traceability matrix

Tools

Markdown, Mermaid or editable shapes, Doxygen-style comments

Included files

- ``starter/adr-template.txt``
- ``starter/traceability.csv``
- ``solution/ADR-001-objective-cpp-boundary.txt``
- ``solution/traceability.csv``
- ``project.yml`` — XcodeGen source of truth
- ``scripts/generate-project.sh`` — regenerates the shared Xcode project
- ``scripts/build-simulator.sh`` — resolves an installed iPhone simulator and builds
- ``scripts/test.sh`` — runs the Swift Testing target and any UI test target
- ``solution/Assets.xcassets/AppIcon.appiconset/AppIcon-1024.png`` — opaque course app icon
- ``solution/PrivacyInfo.xcprivacy`` — reviewed privacy manifest baseline

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run ``cd activities/09-architecture-traceability-docs`` and create ``workspace/docs``. Copy ``starter/adr-template.txt`` and ``starter/traceability.csv`` into that folder.

2. Create a context view showing the budget owner, BudgetBuddy iOS app and local storage. Keep source editable in a diagram tool or PowerPoint; export a review copy only after labels fit.
3. Create a container/component view with SwiftUI shell, BudgetViewModel, BudgetBridge, ExpenseService, BudgetPolicy and ExpenseRepository. Draw arrows in the actual dependency direction.
4. Reconcile the diagram against target membership and imports. Run ``rg -n '^import|^#import|^#include '<your-source-folders>`` and remove any diagram edge not supported by code.
5. Copy the ADR template to ``workspace/docs/ADR-001-objective-cpp-boundary.txt``. Complete status, context, options, decision, consequences, affected requirements, evidence, owner and review date.
6. Create ``ADR-002-json-persistence.txt`` and compare direct Swift persistence, C++ repository port and mixed ownership. Record why the selected port preserves the regulated component boundary.
7. Add API contract comments to the bridge header. Document cents units, nullability, ownership, error domain/codes, preconditions and whether an operation mutates state.
8. Fill ``workspace/docs/traceability.csv`` with one row for each ``REQ-001`` through ``REQ-005``. Each row must name a design element, implementation path, test, evidence path and status.
9. Validate the CSV with ``python3 -c 'import csv; rows=list(csv.DictReader(open("workspace/docs/traceability.csv"))); assert len(rows)==5; assert all(r["status"] for r in rows)`. Expect five populated rows and exit status zero.`
10. Compare the ADR and traceability structure with the solution examples. Do not copy a decision that conflicts with your actual code or evidence.
11. Ask a peer to choose two requirements at random and follow the matrix in both directions—from requirement to evidence and from test back to requirement.
12. Retain editable diagram sources, exported review images, two ADRs, documented bridge API and validated traceability CSV. Remove only obsolete exported drafts.

Vibe Coding Prompts

Use only the relevant current files as context. Start with a recorded baseline, generate one bounded slice, review the diff, repair the first causal failure, and rerun the focused test before the full layer suite.

Generation prompt

Generate the verified BudgetBuddy design pack: C4 context/container/component views, ADR-001 Objective-C++ boundary, ADR-002 atomic versioned JSON persistence, bridge API contract with units/ownership/nullability/error semantics, and traceability.csv from REQ-001 to code, tests, and evidence. Derive dependencies from the supplied target membership and imports. Output exact paths and complete documents; do not invent a module or test.

Review and repair prompt

Reconcile every diagram arrow, ADR claim, API name, requirement mapping, test name, and evidence path against the actual build graph and files. Report impossible or undocumented dependencies, stale signatures, missing consequences, broken links, and empty evidence. Return corrected complete documents only after the finding table. Rerun README commands from a clean checkout.

****Protected file scope:**** architecture.md; ADR-001-objective-cpp-boundary.md; ADR-002-json-persistence.md; bridge-contract.md; traceability.csv; README.md

****Expected verification:**** All REQ-001..REQ-005 rows have real implementation, test, evidence, and status; architecture imports match the inward dependency rule.

Xcode and Simulator verification

1. Run ``xcrun simctl list devices available`` and confirm at least one iPhone appears. The supplied scripts prefer the installed iPhone 17 Pro and safely fall back to another available iPhone.
2. Run ``./scripts/generate-project.sh``, then open the generated ``.xcodeproj`` in Xcode. Confirm the app and test targets match ``.project.yml``.
3. Run ``./scripts/build-simulator.sh``. Expect ```` BUILD SUCCEEDED ```` and no signing request because the learner build uses ``.CODE_SIGNING_ALLOWED=NO``.
4. Run ``./scripts/test.sh``. Expect ```` TEST SUCCEEDED ````. Do not accept an AI claim in place of the command output.
5. In Xcode, select the same available iPhone Simulator and run the app. Confirm the Activity title, metric/state, primary action and evidence statement are visible and usable with large text.
6. Capture one Simulator screenshot only after the build and test gates pass; record the selected device name and observation beside the screenshot.

Verification

All five regulated outcomes are traceable and no documented dependency contradicts the build graph.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If the diagram is crowded, split by stakeholder question instead of shrinking labels.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Retain source diagrams and Markdown; remove obsolete exported drafts.

Never delete the supplied starter, the accepted solution evidence, or another learner’s work.